4. Implement Document Storage and Top-K Retrieval Using a Vector Database

Aim

To implement document storage using a FAISS vector database and retrieve the top-K most relevant documents based on the similarity between query and document embeddings.

Algorithm

1. Import the required libraries, including FAISS, NumPy and Sentence Transformers.
2. Create a collection of sample documents.
3. Load a pre-trained Sentence Transformer model.
4. Generate numerical embeddings for all documents.
5. Normalize the document embeddings for cosine similarity calculation.
6. Create a FAISS vector index using the embedding dimension.
7. Store the document embeddings in the FAISS index.
8. Accept a search query and the required top-K value from the user.
9. Generate and normalize the query embedding.
10. Retrieve the top-K most similar documents from the vector database.
11. Display the retrieved documents and their similarity scores.

Code

```
# Install required libraries
!pip install -q sentence-transformers faiss-cpu

# Import libraries
import faiss
import numpy as np
from sentence_transformers import SentenceTransformer

# Sample documents
documents = [
    "Artificial Intelligence makes machines intelligent.",
```

```
    "Machine Learning helps computers learn from data.",
    "Deep Learning uses artificial neural networks.",
    "Python is a programming language.",
    "Natural Language Processing understands human language.",
    "Computer Vision helps machines analyze images.",
    "Cloud Computing provides online services."
]

# Load embedding model
model = SentenceTransformer('all-MiniLM-L6-v2')

# Generate document embeddings
embeddings = model.encode(
    documents,
    convert_to_numpy=True
)

# Convert to float32
embeddings = np.asarray(embeddings, dtype=np.float32)

# Normalize embeddings
faiss.normalize_L2(embeddings)

# Create FAISS vector database
dimension = embeddings.shape[1]
index = faiss.IndexFlatIP(dimension)

# Store document vectors
index.add(embeddings)
```

```
print("Documents stored successfully!")
print("Total documents:", index.ntotal)

# Get query from user
query = input("\nEnter your search query: ")

# Get top-K value
k = int(input("Enter number of results (K): "))

# Validate K
if k <= 0:
    print("K must be greater than zero.")
else:
    k = min(k, index.ntotal)

    # Generate query embedding
    query_vector = model.encode(
        [query],
        convert_to_numpy=True
    )

    query_vector = np.asarray(
        query_vector, dtype=np.float32
    )

    # Normalize query embedding
    faiss.normalize_L2(query_vector)
```

```
# Retrieve top-K documents
scores, indices = index.search(query_vector, k)

# Display results
print("\nTOP-K DOCUMENT RETRIEVAL")

for rank, (idx, score) in enumerate(
    zip(indices[0], scores[0]), start=1
):
    print("\nRank:", rank)
    print("Document:", documents[idx])
    print("Similarity Score:", round(float(score), 4))
```

Output:

```
Loading weights: 100%  103/103 [00:00<00:00, 2770.35it/s]
FAISS Vector Database Created Successfully!
Total Documents: 7

Enter your search query: How can computers understand human language?

SIMILARITY-BASED DOCUMENT RETRIEVAL

Rank: 1
Document: Natural Language Processing understands human language.
Cosine Similarity: 0.615

Rank: 2
Document: Computer Vision helps machines understand images.
Cosine Similarity: 0.4898

Rank: 3
Document: Machine Learning allows computers to learn from data.
Cosine Similarity: 0.4545
```

Result

Thus, documents were successfully stored in a FAISS vector database, and the top-K relevant documents were retrieved using similarity search.